

Cloud-Deployed Multi-Agent RAG System with ReAct Reasoning, Multi-Tool Calling, and Autonomous Orchestration

Problem Statement

Modern AI applications increasingly require systems that can reason step-by-step, retrieve external knowledge, invoke multiple tools, coordinate between agents, and operate reliably in real-world environments. Single-agent or single-tool systems struggle with complex tasks that involve planning, decision-making, tool selection, and dynamic information retrieval. Additionally, many prototypes fail to move beyond local execution and lack real deployment readiness.

The problem is to design and deploy a **production-oriented AI system** that combines **RAG**, **multi-agent ReAct-style reasoning**, and **multi-tool calling**, orchestrated through a controlled workflow and deployed on a cloud platform. The system must intelligently decide *when to retrieve knowledge*, *when to reason*, *when to call tools*, and *how to synthesize results*, while remaining scalable, auditable, and user-facing.

Project Overview

This project involves building a **cloud-deployed, multi-agent AI system** that uses a **RAG Agent as the core knowledge layer**, combined with **ReAct-based agents** capable of reasoning and invoking multiple tools dynamically. The agents collaborate to solve complex user queries that require planning, information retrieval, external API calls, and structured output generation.

The system is orchestrated using **lightweight LangGraph nodes** (or equivalent orchestration), supports **multi-tool calling** (search, APIs, calculators, databases, etc.), and exposes functionality through a web interface. The final system is deployed on a cloud platform such as **Render**, **Railway**, or **any equivalent service**, making it accessible as a real application rather than a local demo.

Task Overview

- **Task 1:** System architecture and agent role design

- **Task 2:** RAG Agent implementation (knowledge ingestion + retrieval)
- **Task 3:** Multi-agent ReAct reasoning and tool selection
- **Task 4:** Multi-tool integration and execution control
- **Task 5:** Agent orchestration using LangGraph
- **Task 6:** User interface and API layer
- **Task 7:** Cloud deployment and production readiness

Mark / Weight Distribution

Task / Criteria	Weight
RAG Agent Design & Grounding Quality	20%
Multi-Agent ReAct Reasoning	20%
Multi-Tool Calling & Integration	20%
Orchestration & System Architecture	15%
Deployment & Production Readiness	15%
Documentation & Demonstration	10%
Total	100%

Detailed Task-wise Explanation

Task 1: System Architecture and Agent Design

Objective:
Define how agents collaborate and divide responsibilities.

What the Learner Must Do:

- Design multiple agents (e.g., RAG Agent, Planner Agent, Tool-Executor Agent, Synthesizer Agent)
- Clearly define agent responsibilities and interaction flow

Constraints / Guardrails:

- Each agent must have a single, well-defined role
 - No monolithic “do-everything” agent
-

Task 2: RAG Agent Implementation

Objective:

Provide grounded knowledge access to the system.

What the Learner Must Do:

- Ingest domain documents into a vector database
- Implement a RAG agent responsible only for retrieval and grounding
- Return context with citations

Constraints / Guardrails:

- All factual claims must originate from retrieved content
 - Explicit fallback if retrieval fails
-

Task 3: Multi-Agent ReAct Reasoning

Objective:

Enable step-by-step reasoning and decision-making.

What the Learner Must Do:

- Implement ReAct-style reasoning (Reason → Act → Observe)

- Allow agents to decide whether to reason, retrieve, or call a tool
- Maintain intermediate reasoning state

Constraints / Guardrails:

- Reasoning steps must be traceable
 - Tool calls must be justified by reasoning
-

Task 4: Multi-Tool Calling and Control

Objective:

Allow agents to interact with multiple external tools.

What the Learner Must Do:

- Integrate tools such as search APIs, weather APIs, calculators, databases, or custom functions
- Enable dynamic tool selection based on agent reasoning

Constraints / Guardrails:

- Tool access must be whitelisted
 - Invalid or excessive tool calls must be blocked
-

Task 5: Agent Orchestration Using LangGraph

Objective:

Control execution flow and state management.

What the Learner Must Do:

- Use LangGraph lightweight nodes to orchestrate agent interactions
- Manage shared state across agents

- Define entry, transition, and termination conditions

Constraints / Guardrails:

- Execution flow must be deterministic
 - Infinite loops must be prevented
-

Task 6: User Interface and API Layer

Objective:

Expose the system to users.

What the Learner Must Do:

- Build a UI using Gradio or a REST API using Flask/FastAPI
- Display final answers along with reasoning summaries and sources

Constraints / Guardrails:

- UI must remain responsive
 - Errors must be handled gracefully
-

Task 7: Cloud Deployment and Production Readiness

Objective:

Deploy the system as a real-world application.

What the Learner Must Do:

- Containerize or configure the application for deployment
- Deploy on Render, Railway, or any cloud platform
- Manage environment variables, API keys, and secrets

Constraints / Guardrails:

- Application must be publicly accessible
 - Secrets must not be hard-coded
 - System must restart cleanly
-

Final Learner Deliverables

Deliverable	Description
Source Code Repository	Production-ready project implementing a cloud-deployed multi-agent RAG system with ReAct-style reasoning and tool calling
Dataset Files	Domain documents used for knowledge ingestion and retrieval
Vector Database	FAISS or Chroma-based vector store containing indexed documents for grounded retrieval
Working Application Interface	Web-based UI or REST API (Gradio / Flask / FastAPI) exposed via a public cloud deployment
Technical Report	4–6 page report detailing system architecture, multi-agent reasoning, orchestration workflow, and deployment setup
Business Presentation	2-page document or 6–8 slide executive presentation highlighting scalability, production readiness, and use cases
HTML Notebook Export	Final executed notebook demonstrating experiments, validation, or reasoning workflows, exported in HTML format